ASSIGNMENT - 1 Natural Language Processing

By *Mohd Fuzail* Submitted to *Prof. S . Chakrabrati*

Github_Repository : https://github.com/Fuzail1929/NLP_Assignment1.git

Part 1: Sentence Segmentation

Answer -1.1)

A simple top-down approach to sentence segmentation would be to divide the text into sentences by splitting whenever sentence boundary punctuation marks like `'.'` , `'!'` or `'?'`, `etc` are encountered in the text .

Example :

Input:

" Dr. Ghanshyam went to the clinic . He was cheerful ! "

Output:

["Dr", "Ghanshyam went to the clinic ", "He was cheerful"]

Issues with this Approach

Although computationally efficient, this method fails in several real-world situations:

- **Abbreviations:**
  Example: *Dr.*, *Prof.*, *etc.*
  Possible Reason : The period does not indicate sentence ends.
- **Decimal numbers , date ,time:**
  Example: *it can split 3.14 which is π , Date-> 07.03.2026 will have the same faith.*
- **Acronyms and initials:**
  Example: It will fail for Acronyms like *U.S.A.* , names like *A. P. J. Abdul Kalam* .
- **Ellipsis (`...`)** can create ambiguity in sentence segmentation because it may indicate hesitation, continuation, or emphasis rather than the end of a sentence
  Example: *ohh… What an innings by Virat kohli ? such exclamatory sentences could face the issue .*

- **Web URLs / Emails:**
  Example: It will make *ch22b080@smail.iitm.ac.in -> "ch22b080@smail" , "iitm","ac" ,"in"*

Possible Strategies :

1. **Abbreviation handling:** Maintain a list of common abbreviations (e.g., *Dr.*,Prof. *etc.*) to avoid incorrect splitting.
2. **Context-based rules:** Sentence boundaries usually follow syntactic patterns, and local context provides strong clues for disambiguation.

   Check the surrounding words *(capitalization, spacing, sentence structure*) before deciding a boundary.

3. **Pattern recognition:** Numbers follow predictable formats, allowing simple regex-based rules to reduce errors significantly

   ->So, detect decimals, dates, acronyms, and URLs so that punctuation inside the sentence is not treated as sentence endings.

4. **Statistical methods:** Data-driven methods generalize better and adapt to unseen cases without manually adding rules.

   ->Use data-driven models like Punkt that learn sentence boundary probabilities from text.

5. **Linguistic analysis:** Linguistic information helps distinguish punctuation used inside phrases from true sentence endings, improving accuracy on complex text.

   ->Use dependency parsing or POS tagging to identify complete clauses and combine punctuation with syntactic structure.

6. **Hybrid approach:** A hybrid system balances computational efficiency and segmentation accuracy, making it suitable for real-world NLP pipelines.

   -> Combine rule-based(it is fast), statistical(more adaptive), and linguistic methods(more accurate) for better accuracy and robustness.

Answer - 1.2)

A simple **top-down (rule-based) sentence segmentation** approach splits text whenever punctuation marks such as '! ' or '? ' appears. It is fast and easy to implement but fails when punctuation is used for other purposes.
Example: *"Prof. V Kamakoti delivered a Speech on Republic Day "* → a naive method may incorrectly split after '*Prof*'.

NLTK Punkt Sentence Segmenter uses a ***statistical, unsupervised approach*** that learns abbreviation patterns and sentence boundary probabilities from data. It uses contextual cues to distinguish true sentence endings from abbreviations.
Example: " *Dr. Sachin went to Clinic "* → Punkt correctly treats *Dr.* as an abbreviation and avoids incorrect splitting.

spaCy's sentence segmentation uses ***linguistic and syntactic information*** such as token attributes and dependency parsing to identify sentence boundaries based on grammatical structure. This makes it more reliable for complex text.
Example: *"Although he was tired, he continued working. Later, he stopped."* → spaCy correctly identifies sentence boundaries using syntactic context.

Approach	***Method***	***Strength***	***Limitation***
Naive (Rule-based)	Simple Punctuation rules	Fast ,simple	It can't handle Abbreviation
NLTK Punkt	Statistical learning	Handles abbreviations & context	Limited lingusitic Understanding
spaCy	Uses linguistic and syntactic information	High accuracy, syntax-aware	It is Computationally heavy approach

---

Answer- 2)

a) **15 sentences that are likely to break a naive segmentationSystem :**
   1. The software version is 3.2.10-beta.
   2. What?! That's unbelievable!
   3. Capt. Monkey D. Luffy led the mission successfully.
   4. The meeting starts at 9.45 a.m. tomorrow.
   5. "Go now!" he yelled loudly.

6. The rocket reached Mach 3.7 during testing
7. Hmm… I am not sure about that.
8. Mr. Will Smith arrived at 6 p.m. and left early.
9. Stop!!! This is dangerous.
10. The U.S. economy grew more than 2.5% than the U.K. economy this year .
11. He whispered, “Don’t move.” before leaving.
12. Contact me at Fuzail1929@gmail.com immediately.
13. She bought 1.25 kg of oranges today.
14. Are you serious?! I don’t think so.
15. Well… maybe we should wait.

b) Categorization by Error Type :

Sentence No.	Sentence (Short Description)	Error Type
1	Software version 3.2.10-beta	Version numbers / Dots inside tokens
2	What?! That’s unbelievable!	Multiple punctuation
3	Capt. Monkey D. Luffy	Abbreviations / Initials
4	9.45 a.m.	Time expression + abbreviation
5	“Go now!”	Quotes
6	Mach 3.7	Decimal numbers
7	Hmm… I am not sure	Ellipsis
8	Mr. Will Smith at 6 p.m.	Abbreviations + time
9	Stop!!!	Multiple punctuation
10	U.S. and U.K. economy	Acronyms with periods
11	“Don’t move,”	Quotes + punctuation
12	Email address	Email / special characters
13	1.25 kg	Decimal numbers
14	Are you serious?!	Multiple punctuation
15	Well… maybe we should wait	Ellipsis

c)

Segmenter	Errors	Typical Failure Cases
Naive (Rule-based)	**8**	Abbreviations, decimals, ellipsis, acronyms
NLTK Punkt	**3**	Multiple punctuation and informal expressions
spaCy	**1**	Rare ambiguity in expressive punctuation

The naive segmenter produced the highest number of errors because it relies only on punctuation marks and cannot understand context.

Punkt performed better by learning abbreviation patterns and sentence boundary probabilities, reducing errors related to abbreviations and acronyms.

spaCy achieved the best performance due to its use of linguistic and syntactic information, which allows it to handle complex punctuation and contextual sentence boundaries more accurately.

Part 2: Tokenization

Answer -1) Simplistic Top-Down Approach

A simple top-down tokenization approach splits text based on whitespace and punctuation rules. In this implementation, a regular expression is used to extract sequences of word characters (`\w+`) and to treat punctuation marks as separate tokens. The method does not rely on contextual or linguistic knowledge.

Proposed Method

- Split text using whitespace.
- Separate punctuation marks (`. , ! ? ; :`) as individual tokens.
- Extract continuous alphanumeric sequences as tokens.

Example

Input:

Mr. Priyanshu Verma scored 98.5 marks!

Output :

["Mr", ".", "Priyanshu", "Verma",”scored”, "98", ".", "5", "marks", "!"]

My Approach definitely have limitations like :

- **Abbreviations**: “Mr." incorrectly split into ["Mr", "."]
- **Decimals** : "98.5" split into ["98", ".", "5"]
- **Contractions and Hyphenated words** : don’t and state-of-the-art like words will split unnecessarily
- **Email/URLs** : https://www.google.com
  ->will get split wrongly .

Answer-2)

Penn Treebank Tokenizer (NLTK)

The Penn Treebank tokenizer is a **rule-based, top-down tokenization system** built according to the linguistic conventions of the Penn Treebank corpus. It applies predefined handcrafted rules to handle punctuation, contractions (e.g., *“don’t” → ["do", "n’t"]*), and token boundaries consistently.

Justification:
It is mainly a top-down approach because it applies clearly defined rules to decide where tokens should be split.

However, since these rules are designed based on linguistic corpus annotations, the tokenizer performs more reliably and accurately than a simple naive rule-based method.

---

spaCy Tokenizer

The spaCy tokenizer follows a **hybrid approach**. It combines rule-based prefix, suffix, and infix splitting with special-case rules and linguistic knowledge from its trained language models.

Justification:
Since **,** It combines clear rule-based tokenization with language-aware statistical processing. Because it uses both predefined rules and learned linguistic knowledge, it is considered a hybrid approach and performs well on real-world text

---

Answer -3)

a) Our approach performs better than the Penn Treebank and spaCy tokenizers:

From cranfield dataset .

Phrase:

high-speed aircraft

Naive : ["high-speed", "aircraft"]

Penn Treebank : ["high", "-", "speed", "aircraft"]

spaCy : ["high", "-", "speed", "aircraft"]

In domain-specific technical corpora such as Cranfield, compound terms like *"high-speed aircraft"* represent a single concept.

The **naive tokenizer preserves this compound** as one token , whereas Penn Treebank and spaCy split it around the hyphen . For **information retrieval tasks** involving technical terminology, preserving compound terms may be beneficial .

---

b) Our approach performs worse than the Penn Treebank and spaCy tokenizers

From cranfield dataset ,

Phrase:

prandtl's classical boundary-layer problem

Naive : ["prandtl", "'", "s", "classical", "boundary-layer", "problem"]

Penn Treebank : ["prandtl", "'s", "classical", "boundary", "-", "layer", "problem"]

spaCy : ["prandtl", "'s", "classical", "boundary", "-", "layer", "problem"]

For possessive constructions such as *"Prandtl's classical boundary-layer problem"*, the naive tokenizer splits the apostrophe separately , whereas Penn Treebank and spaCy correctly treat *"'s"* as a single possessive unit .
Hence, **linguistically informed tokenizers** perform better for **grammatical processing.**

Part 3: Stemming and Lemmatization

Answer -1) Stemming :

1. Stemming is a rule-based method that removes common suffixes from words without considering their grammatical meaning.

2. It simply chops off endings to get a root form, which may not always be a valid English word.
3. For example, *"studies"* becomes "studi" and *"running"* becomes "run." Here, *"studi"* is not a valid English word, showing that stemming may distort word forms.

4. Because it uses simple rules, stemming is fast and computationally inexpensive.

Lemmatization:

1. Lemmatization,on the other hand, uses linguistic knowledge such as vocabulary and morphological analysis to return the correct dictionary form (lemma) of a word.

2. For example, *"studies"* becomes "study", and *"running"* becomes "run." The output is usually a meaningful and valid word.

3. It Uses lexical resources(e.g., WordNet) and produces valid dictionary words.

4. However, lemmatization is computationally more expensive because it relies on lexical resources like WordNet.

Answer - 2)

We applied Stemming (Porter Stemmer) and Lemmatization (WordNet Lemmatizer) to the tokenized Cranfield dataset.

a) Changes in vocabulary size :

Porter stemming reduces words more aggressively by truncating suffixes.

For example:

1. *investigation* → **investig**
   boundary → **boundari**

As many related words are reduced to shorter stems, the overall vocabulary size becomes smaller.

Whereas , lemmatization preserves valid base forms:

1. *constructing* → **constructing**
2. *associated* → **associated**

Thus, lemmatization results in a slightly larger but linguistically cleaner vocabulary.

---

b) Examples of Over-Stemming

Porter Stemmer sometimes produces non-dictionary forms:

1. *boundary* → **boundari**
2. *velocity* → **veloc**

These truncated words lose proper spelling and may reduce readability. This is an example of over-stemming.

---

c) Lemmatization Preserving Semantic Meaning

Lemmatization maintains valid dictionary words:

1. *associated* remains **associated**
2. *similarity* remains **similarity**

Unlike stemming, it will not distort words into non-existing forms.

Therefore, lemmatization better preserves semantic meaning and is more suitable for tasks requiring linguistic interpretation.

Part 4: Stopword Removal

Answer -1)

NLTK uses the standard English stopword list to remove Stopwords from the tokenized Cranfield documents using from.
Each token was checked against this list and removed if it matched.
This helps eliminate common words like *"the", "is", "of", "and"*, which appear frequently but add little value in distinguishing documents for retrieval.

Answer -2)

We can identify stopwords directly from the corpus. Instead of using a predefined stopword list, if a word appears in a large proportion of documents, it does not help distinguish one document from another.

To implement this, we compute the document frequency $df(t_i)$ for ith term in corpus and calculate:

$$df(t_i) / |D|$$

If this ratio exceeds a chosen threshold (e.g., 70–80%), the term is treated as a stopword. This approach removes corpus-specific high-frequency words and helps reduce noise while improving retrieval efficiency.

Answer - 3)

We used the bottom-up (data-driven) approach,setting a threshold of 70% and **12 stopwords** were identified from the Cranfield corpus. For comparison, the predefined NLTK English stopword list contains **198 stopwords**.

Out of the 12 corpus-based stopwords, **10 words overlap** with the NLTK stopword list that we observed . This shows that most high-frequency words identified statistically are already recognized as common English stopwords.

Interestingly, the corpus-based list also identified punctuation symbols **such as "," and "."** as high-frequency terms.

These appear in almost every document just like other stopwords and therefore have very low discriminative power in this dataset.

While ,we found many words present in the NLTK list (such as *"**between", "but", "why", "will"***) which were not identified by the corpus-based method. This is

because, although they are common English function words, they do not appear in a very high proportion of documents(more than 70 % as out set threshold) in the Cranfield dataset.

Part 5: Retrieval

Answer -1)

We are given 1. $|Q|$ = number of queries 2. $|D|$ = number of documents
3. $|V|$ = vocabulary size
To calculate the similarity between **every query and every document**, we must compare each query with all documents we need $|Q| \times |D|$ total similarity computations.
If we use cosine similarity with TF-IDF vectors of dimension $|V|$, each having similarity computation takes $O(|V|)$ time.

So that the total time Complexity becomes $O(|Q| \times |D| \times |V|)$.

This is a computationally expensive method because even documents that share no common terms with the query are still getting compared.

Answer -2) Inverted Index Approach :

Construct an inverted index that stores for each term in the vocabulary, a list of documents that contain that term. Now, Instead of comparing a query with all $|D|$ documents, we:

- Will Look at the terms present in the query.
- Try to Retrieve only the documents that contain those terms.
- And finally Compute similarity only for those relevant documents.

Assume that **k** is the average number of non-zero (unique) terms in a query

And **df** is the average document frequency of a term

Then, instead of comparing with all $|D|$ documents, we compare with approximately:

k × df documents.

So the new **time complexity** becomes approximately is

$$\mathbf{O(|Q| \times k \times df)}$$

As $\mathbf{k \times df \ll |D|}$, the inverted index significantly reduces the number of similarity computations so that $O(|Q| \times k \times df)$ is much less than $O(|Q| \times |D| \times |V|)$ i.e

Time Complexity in Inverted Index Approach is less than time required in Time Complexity of naive approach .

Approach	**Time Complexity**
Naive	$O(|Q| \times |D| \times |V|)$
Inverted Index	$O(|Q| \times k \times df)$

The inverted index exploits sparsity in TF-IDF vectors and avoids unnecessary comparisons, making retrieval much more efficient for large document collections.

FIN